



FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
Sistema de Gestión de Historias Clínicas y Adopción para Refugios de
Animales

Autores:

KEVIN JHON FERREL PERALTA - N00356981
HAROLD CRISTHIAN SOVERO REVOLO - N00428298

Curso:

TÉCNICAS DE PROGRAM.ORIE. OBJ.

Docente:

MARTIN EDUARDO TORRES RODRIGUEZ

Los Olivos - Lima – Perú

2026-1

INDICE

Sistema de Gestión de Historias Clínicas y Adopción para Refugios de Animales	1
1. Definición del Problema	3
1.1 Situación problemática	3
1.2 Problema identificado	3
1.3 Diagrama de Ishikawa	3
2. Antecedentes	4
3. Restricciones Realistas	5
3.1 Restricciones técnicas	5
3.2 Restricciones económicas	5
3.3 Restricciones operativas	5
4. Objetivos del Proyecto	6
4.1 Objetivo General	6
4.2 Objetivos Específicos	6
5. Alcance de la Solución	6
5.1 Alcance Funcional	6
5.2 Alcance Técnico	7
5.3 Exclusiones del Proyecto (Fuera de Alcance)	7
6. Requerimientos Funcionales	7
7. Historias de Usuario	9
8. Diagrama de Clases	9
9. Criterios de Aceptación	11
10. Implementación del Proyecto	12
10.1 Tecnologías utilizadas	12
10.2 Manejo de archivos en Java	12
10.3 Funcionalidades implementadas	12
11. Resultados	12
12. Conclusiones	12
13. Bibliografía	13
14. Anexos	13

1. Definición del Problema

1.1 Situación problemática

Actualmente, la gran mayoría de los refugios de animales operan bajo una gestión administrativa empírica. El uso de cuadernos físicos, registros en papel o múltiples hojas de cálculo desconectadas entre sí genera un entorno de trabajo altamente ineficiente y propenso al error humano. Esta fragmentación de la información provoca la pérdida constante de historiales médicos y la duplicidad de registros, lo que se traduce en un desorden operativo en el día a día. Por ejemplo, ante una urgencia veterinaria, el tiempo que el personal invierte buscando físicamente la ficha de un animal retrasa de forma crítica su atención clínica.

Además de la ineficiencia administrativa, este déficit impacta directamente en el bienestar animal y en la transparencia de los procesos. El seguimiento de esquemas de vacunación y tratamientos para enfermedades crónicas se vuelve inmanejable cuando se depende exclusivamente de la memoria del personal o de revisiones manuales. En paralelo, en el área de adopciones, la falta de un registro centralizado dificulta la evaluación del historial de los adoptantes. Esto incrementa el riesgo de entregar una mascota a un perfil no apto o, por el contrario, frustra y retrasa el proceso para familias idóneas debido a la lentitud en la verificación de datos.

La gestión de estos refugios implica procesar un volumen importante de datos críticos. Según los estándares promovidos por organizaciones especializadas como HumanePro, la trazabilidad estructurada de la información —que abarca desde los datos de ingreso y problemas de conducta, hasta el control médico riguroso y los contratos de adopción finales— es una práctica fundamental para la correcta operatividad de estas instituciones.

Por lo tanto, la dependencia de métodos manuales o herramientas ofimáticas básicas ya no es sostenible. Se hace evidente y urgente la necesidad de implementar una solución informática robusta. Un sistema que organice la información de manera lógica y relacional, reduciendo a cero la dependencia del papel, mitigando los errores de digitación y garantizando un control integral que agilice tanto el cuidado médico como la reubicación exitosa de los animales.

1.2 Problema identificado

El problema central radica en la **deficiente gestión de historias clínicas y procesos de adopción de mascotas en refugios de animales**, una situación ocasionada directamente por la dependencia de registros manuales y la carencia absoluta de un sistema de información centralizado.

Esta deficiencia estructural genera un impacto negativo que se evidencia en tres niveles críticos del refugio:

- **Nivel Clínico:** Imposibilidad de mantener una trazabilidad exacta y en tiempo real del estado de salud de cada mascota, lo que provoca la omisión de alertas de vacunación, interrupción de tratamientos y retrasos en la atención veterinaria.
- **Nivel Operativo:** Alta vulnerabilidad a la pérdida de documentos físicos, duplicidad de datos por registros paralelos y cuellos de botella diarios al momento de buscar el expediente de un animal específico.
- **Nivel Administrativo (Adopciones):** Ausencia de filtros rápidos y seguros para cruzar datos y evaluar el historial de los solicitantes, lo que ralentiza la reubicación de las mascotas e incrementa el riesgo de entregarlas a perfiles no idóneos.

Formulación del problema: ¿De qué manera el desarrollo de un sistema informático de escritorio en Java, estructurado bajo los pilares de la Programación Orientada a Objetos, logrará optimizar la gestión de historias clínicas y asegurar la trazabilidad en los procesos de adopción dentro de los refugios de animales?

1.3 Diagrama de Ishikawa



2. Antecedentes

HumanePro (s. f.): El sustento teórico sobre qué información es crítica dentro de un refugio no puede dejarse a la especulación. Esta organización establece con claridad que los grupos de rescate deben registrar de forma estricta y exacta variables como la especie, el historial médico detallado, el control de vacunas, los problemas de conducta y el estado de las solicitudes de adopción. Si trasladamos este marco operativo al desarrollo de software, este antecedente constituye la base para el modelado de dominio de nuestro sistema. No es una simple lista de datos; determina directamente los atributos y el encapsulamiento de clases fundamentales como Mascota y Adopcion. Al definir el estado y comportamiento de estos objetos basándonos en este estándar, nos aseguramos desde el código de mitigar la redundancia de datos y estructurar un sistema con lógica de negocio real.

Association of Shelter Veterinarians (2022): La rigurosidad en la gestión de un refugio va más allá de lo puramente administrativo; es un factor crítico para la supervivencia y bienestar animal. En sus Guidelines for Standards of Care in Animal Shelters, la asociación determina que la existencia de registros médicos estructurados y organizados es una práctica obligatoria para garantizar una atención veterinaria responsable y un seguimiento clínico eficaz. Para nuestro proyecto en Java, este antecedente proporciona la justificación técnica de la arquitectura y modularidad del software. Nos obliga a aplicar el principio de responsabilidad única en POO: la salud del animal no puede ser un atributo secundario pegado de forma caótica en cualquier lado; exige la creación de módulos y clases totalmente independientes para HistorialMedico, Vacuna y Tratamiento. Esto garantiza un acoplamiento débil entre clases, permitiendo que las consultas y actualizaciones médicas sean eficientes y seguras.

Capterra (2026) y Animals First (s. f.): El análisis del estado del arte tecnológico demuestra que la centralización de la información es el estándar de la industria. Las plataformas evaluadas en Capterra y soluciones comerciales vigentes como Animals First comprueban que unificar los expedientes clínicos y los flujos de adopción en un único ecosistema digital es la única estrategia que optimiza los tiempos de respuesta y elimina el papeleo operativo. Estos referentes comerciales respaldan directamente la viabilidad operativa de nuestra propuesta. Aunque nuestro sistema se diseñe como una aplicación de escritorio local en Java debido a las restricciones académicas del curso, la lógica funcional interna replica el comportamiento de las herramientas líderes del mercado. Esto valida que el

uso de colecciones de objetos y la persistencia de datos mediante el manejo de archivos en Java no es un ejercicio teórico aislado, sino una solución escalable alineada a las necesidades reales del sector.

3. Restricciones Realistas

3.1 Restricciones técnicas

Desarrollo nativo en Java SE y Eclipse: El ecosistema de desarrollo se restringirá estrictamente al uso de Java Standard Edition (SE) bajo el entorno del IDE Eclipse. Esta delimitación técnica garantiza que la arquitectura del software se construya aplicando los fundamentos puros de la Programación Orientada a Objetos (encapsulamiento, herencia y polimorfismo) desde cero. Se prohíbe el uso de frameworks empresariales de alto nivel que abstraigan u oculten la lógica estructural que este proyecto busca evidenciar.

Persistencia mediante flujos nativos (I/O y Serialización): Alineado al alcance del curso, el sistema operará de manera independiente sin la integración de motores de bases de datos relacionales (como MySQL o SQL Server). La persistencia y recuperación de la información médica y administrativa se gestionará exclusivamente a través del manejo de archivos nativo de Java (texto plano y serialización de objetos). Esta restricción es clave para demostrar un dominio técnico sobre el ciclo de vida de los objetos en memoria y su correcta conversión para el almacenamiento local continuo.

Gestión de configuración y versiones: La trazabilidad del código fuente, la auditoría de cambios y el trabajo colaborativo se ejecutarán de manera innegociable a través de Git y el repositorio remoto en GitHub. No se validará la integración de código por medios no oficiales; todo avance debe estar respaldado por commits descriptivos, manejo de ramas (branches) y resolución de conflictos, aplicando buenas prácticas de ingeniería de software desde la primera línea de código.

3.2 Restricciones económicas

Presupuesto cero y viabilidad Open Source: La ejecución, desarrollo y pruebas del proyecto están estrictamente condicionados a una inversión financiera nula (presupuesto \$0.00). Esta restricción económica impacta directamente en la arquitectura de la solución, ya que nos obliga a descartar la adquisición de componentes de software de terceros, librerías comerciales para interfaces gráficas avanzadas o servicios de almacenamiento y despliegue en la nube. En consecuencia, todo el ciclo de vida del software se sustentará exclusivamente en herramientas de código abierto (Open Source) y tecnologías bajo licencias educativas gratuitas (Java SE, Eclipse IDE, Git). Esto demuestra la capacidad del equipo para construir una solución funcional, robusta y completa, gestionando eficientemente los recursos gratuitos disponibles sin incurrir en costos operativos o de infraestructura para el refugio.

3.3 Restricciones operativas

Limitante temporal estricta (Timeboxing): El cronograma de ingeniería y desarrollo está circunscrito exclusivamente a las semanas de duración del presente ciclo académico. Esta restricción cronológica nos obliga a congelar el alcance del proyecto una vez definidos los 40 requerimientos funcionales críticos. Cualquier característica, módulo adicional o mejora visual que escape de este núcleo base será catalogada como "fuera de alcance" para evitar la corrupción del proyecto (scope creep), asegurando así un entregable 100% funcional y testeado para la fecha de sustentación.

Entorno de ejecución aislado (Standalone): La solución técnica está concebida de manera estricta como una aplicación de escritorio local. Por diseño y por las limitaciones operativas del refugio, no contempla un modelo de arquitectura cliente-servidor, despliegue en entornos web ni acceso remoto

mediante red. Toda la estructura del sistema —la interfaz gráfica, la lógica de negocio basada en POO y la lectura/escritura de los archivos de persistencia— se ejecutará de forma monolítica y offline en una única terminal física. Esto garantiza la operatividad continua del refugio sin depender de una conexión a internet estable.

4. Objetivos del Proyecto

4.1 Objetivo General

Desarrollar un sistema informático de escritorio en Java basado en los pilares fundamentales de la Programación Orientada a Objetos (POO), con la finalidad de centralizar, automatizar y optimizar la gestión de historias clínicas veterinarias y la trazabilidad de los procesos de adopción dentro de los refugios de animales.

Este objetivo se desglosa en tres componentes metodológicos:

- **El Qué:** Un sistema de software de escritorio interactivo y seguro.
- **El Cómo:** Aplicando de forma estricta los conceptos de modularidad, encapsulamiento, relaciones entre clases (agregación y composición) y persistencia a través de archivos nativos en Java.
- **El Para qué:** Eliminar la deficiencia operativa de los registros manuales, garantizando un control clínico riguroso y acelerando la reubicación segura de las mascotas en hogares idóneos.

4.2 Objetivos Específicos

Estructurar la arquitectura del sistema mediante un modelo de clases cohesivo y desacoplado:

Diseñar e implementar las entidades principales del dominio (Mascota, Adoptante, CitaMedica, Tratamiento, Vacuna, Adopcion) aplicando rigurosamente el principio de encapsulamiento. El objetivo es garantizar una alta cohesión (que cada clase maneje solo la información y lógica que le corresponde) y un bajo acoplamiento (minimizar la dependencia entre clases), lo que asegura un código escalable, libre de redundancias y fácil de mantener.

Garantizar el estado y la disponibilidad de la información mediante persistencia nativa:

Implementar el manejo de archivos (E/S de flujos de texto o serialización de objetos) en Java para asegurar que los registros médicos y administrativos no se pierdan al finalizar la ejecución del programa. Este objetivo evidencia el dominio técnico sobre el ciclo de vida de los objetos, garantizando la continuidad operativa del refugio sin depender de motores de bases de datos externos.

Desarrollar una Interfaz Gráfica de Usuario (GUI) orientada a eventos para la gestión operativa:

Construir una capa de presentación intuitiva y ergonómica que permita al personal del refugio ejecutar eficientemente las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en todos los módulos. Este desarrollo mantendrá una estricta separación estructural entre la lógica de negocio (clases POO) y la vista, gestionando la interacción del usuario mediante *listeners* y el control de excepciones para evitar cierres inesperados del sistema.

5. Alcance de la Solución

El alcance del proyecto define los límites funcionales y técnicos del sistema, garantizando que la solución responda directamente a la problemática identificada sin comprometer los tiempos de entrega del ciclo académico.

5.1 Alcance Funcional

El sistema automatizará el ciclo de vida operativo de las mascotas desde su ingreso al refugio hasta su adopción definitiva, estructurándose en cuatro módulos core:

- **Módulo de Registro de Mascotas:** Gestionará el alta, modificación y baja lógica de los animales. Capturará datos de identidad (código único, nombre, especie, raza, edad estimada), descriptivos (física y conductual) y de estado, permitiendo clasificar de forma dinámica si el animal está en cuarentena, disponible para adopción, en tratamiento o ya adoptado.
- **Módulo de Historial Médico:** Centralizará el seguimiento clínico veterinario. Permitirá la creación

de expedientes médicos asociados a cada mascota, registrando de forma cronológica las citas de revisión (con métricas de peso y diagnóstico), la programación y aplicación de vacunas, y la gestión de tratamientos específicos para enfermedades detectadas.

- **Módulo de Gestión de Adoptantes:** Administrará el directorio de personas interesadas en adoptar. Registrará datos de contacto, filtros de validación socioeconómica o de vivienda, y las preferencias de búsqueda del adoptante (especie, tamaño o edad de la mascota), agilizando el cruce de información.
- **Módulo de Proceso de Adopción:** Controlará el flujo de emparejamiento y legalización. Evaluará el estado de las solicitudes, vinculará formalmente un adoptante aprobado con una mascota disponible, registrará las fechas de entrega y permitirá programar alertas para las visitas de seguimiento post-adopción, incluyendo la reversión del proceso si el bienestar del animal se ve comprometido.

5.2 Alcance Técnico

- **Arquitectura de Escritorio (Standalone):** La aplicación será construida con una interfaz gráfica nativa (GUI) orientada a eventos. Toda la lógica de negocio y las interfaces correrán de manera local en el equipo del refugio.
- **Persistencia en Archivos:** El almacenamiento de datos no dependerá de servidores externos. Se utilizarán flujos de entrada y salida nativos de Java (Manejo de archivos de texto o serialización binaria de colecciones de objetos) para guardar los estados de las clases de forma local.

5.3 Exclusiones del Proyecto (Fuera de Alcance)

Para evitar la sobrecarga en el desarrollo y mantener el enfoque estricto en las técnicas de POO exigidas por el curso, quedan completamente descartados los siguientes componentes:

- **Módulo de Contabilidad y Finanzas:** No se gestionarán balances, flujos de caja, ni presupuestos operativos del refugio.
- **Pasarelas de Pago:** El sistema no procesará donaciones económicas directas, transacciones en línea ni cobros por adopción.
- **Control de Inventario de Alimentos e Insumos:** No se realizará la trazabilidad de stock de comida, medicamentos en bodega ni donaciones materiales.

6. Requerimientos Funcionales

Para el desarrollo del sistema se han definido 40 requerimientos funcionales críticos, estructurados modularmente para cubrir todas las reglas de negocio del refugio:

Módulo 1: Gestión de Mascotas

- **RF01:** El sistema debe permitir registrar una nueva mascota ingresando un código único, nombre, especie, raza y fecha de rescate.
- **RF02:** El sistema debe permitir modificar los datos descriptivos de una mascota ya registrada.
- **RF03:** El sistema debe permitir cambiar el estado operativo de la mascota (Disponible, En cuarentena, En tratamiento, Adoptada).
- **RF04:** El sistema debe permitir dar de baja lógica a un registro en caso de fallecimiento del animal.
- **RF05:** El sistema debe permitir la búsqueda de una mascota específica utilizando su código identificador o nombre.

- **RF06:** El sistema debe generar un listado de todas las mascotas registradas.
- **RF07:** El sistema debe filtrar y listar únicamente las mascotas que tengan el estado "Disponible" para adopción.
- **RF08:** El sistema debe permitir registrar una descripción física detallada (color, tamaño, marcas particulares).
- **RF09:** El sistema debe permitir registrar observaciones de problemas de conducta al momento del ingreso.
- **RF10:** El sistema debe registrar de forma automática la fecha del sistema como fecha de ingreso de la mascota.

Módulo 2: Historial Médico Veterinario

- **RF11:** El sistema debe permitir crear una nueva cita médica asociada al código de una mascota.
- **RF12:** El sistema debe permitir registrar el peso y la talla del animal en cada cita generada.
- **RF13:** El sistema debe permitir ingresar el diagnóstico emitido por el veterinario en la cita.
- **RF14:** El sistema debe permitir cancelar o reprogramar una cita médica en estado pendiente.
- **RF15:** El sistema debe mostrar el historial cronológico de todas las citas de una mascota.
- **RF16:** El sistema debe permitir registrar el inicio de un tratamiento médico especificando la duración.
- **RF17:** El sistema debe permitir actualizar el estado del tratamiento (En curso, Suspendido, Finalizado).
- **RF18:** El sistema debe permitir asociar una o más medicinas prescritas al tratamiento.
- **RF19:** El sistema debe permitir registrar la aplicación de una vacuna, incluyendo el nombre y laboratorio.
- **RF20:** El sistema debe permitir programar la fecha de la próxima dosis de una vacuna.
- **RF21:** El sistema debe generar un listado de vacunas pendientes filtrado por mascota.
- **RF22:** El sistema debe permitir exportar o visualizar un resumen consolidado del historial clínico del animal.

Módulo 3: Gestión de Adoptantes

- **RF23:** El sistema debe permitir registrar un nuevo adoptante ingresando DNI, nombres completos, edad y teléfono.
- **RF24:** El sistema debe validar que el DNI ingresado no se encuentre duplicado en el registro.
- **RF25:** El sistema debe permitir actualizar los datos de contacto y dirección del adoptante.
- **RF26:** El sistema debe permitir registrar las preferencias de adopción (ej. prefiere perro pequeño o gato adulto).
- **RF27:** El sistema debe permitir registrar el tipo de vivienda del adoptante (casa propia, departamento, alquilado).
- **RF28:** El sistema debe permitir asignar un estado de validación al adoptante (En evaluación, Aprobado, Rechazado).
- **RF29:** El sistema debe requerir un motivo obligatorio si el estado del adoptante cambia a "Rechazado".
- **RF30:** El sistema debe permitir buscar el perfil de un adoptante usando su número de DNI.
- **RF31:** El sistema debe listar todos los adoptantes que tengan el estado "Aprobado".

Módulo 4: Proceso de Adopción

- **RF32:** El sistema debe permitir generar una nueva solicitud de adopción vinculando un DNI

aprobado con el ID de una mascota disponible.

- **RF33:** El sistema debe bloquear automáticamente la disponibilidad de una mascota una vez que entra en un proceso de adopción.
- **RF34:** El sistema debe permitir registrar la fecha de aprobación y entrega formal de la mascota.
- **RF35:** El sistema debe cambiar el estado de la mascota a "Adoptada" tras concretar la vinculación.
- **RF36:** El sistema debe permitir programar hasta dos fechas para visitas de seguimiento post-adopción.
- **RF37:** El sistema debe permitir registrar las observaciones recabadas durante las visitas de seguimiento.
- **RF38:** El sistema debe permitir ejecutar una reversión de adopción, devolviendo la mascota al estado "Disponible".

Módulo 5: Seguridad y Sistema

- **RF39:** El sistema debe requerir un nombre de usuario y contraseña para conceder acceso a los módulos.
- **RF40:** El sistema debe permitir cerrar la sesión activa del usuario para garantizar la seguridad de los registros.

7. Historias de Usuario

Para mantener un enfoque centrado en el usuario y garantizar la trazabilidad del proyecto, se han definido las siguientes historias de usuario alineadas a los requerimientos funcionales críticos:

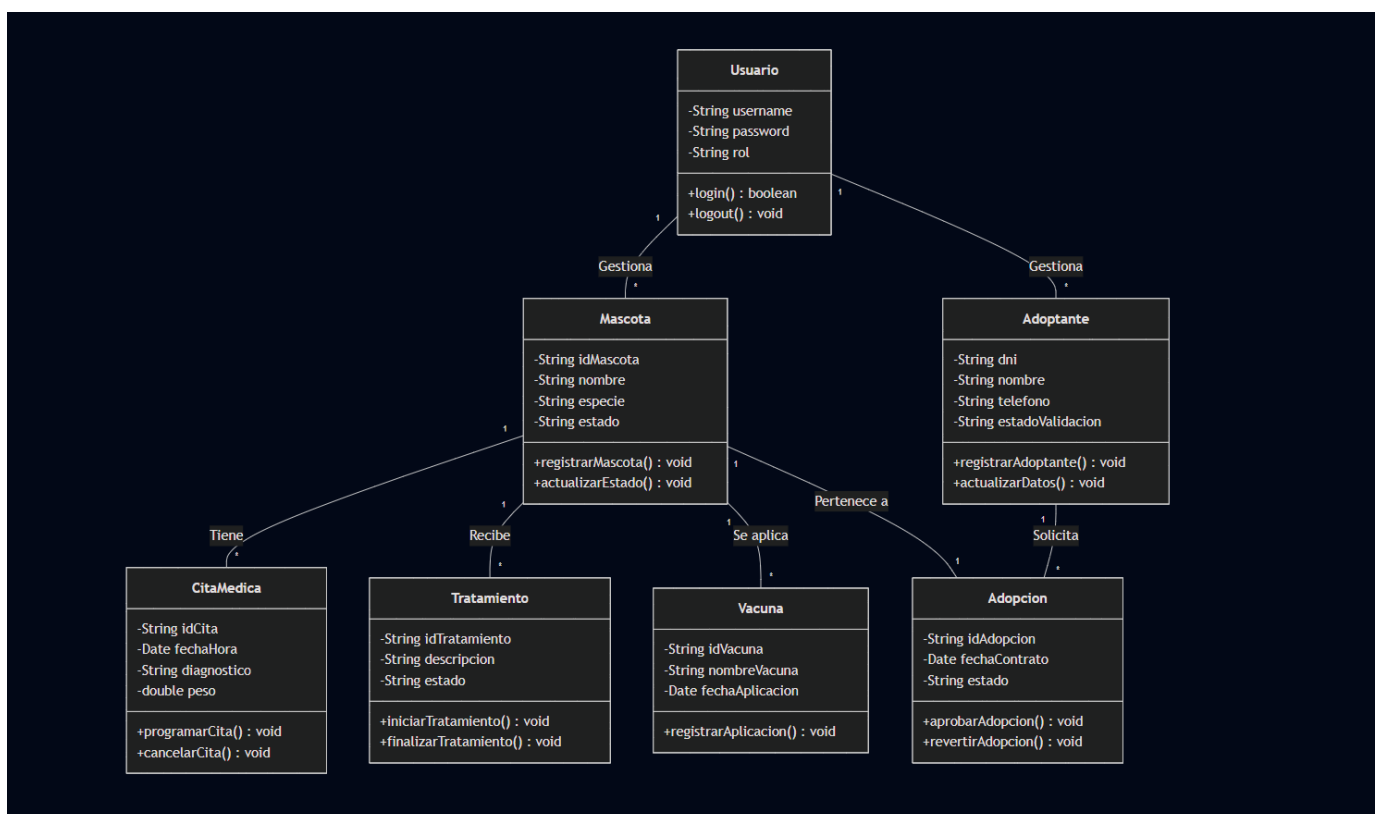
- **HU01:** Como *Personal de Recepción*, quiero registrar una nueva mascota para ingresarla al sistema centralizado del refugio. **(Asociado a RF01)**
- **HU02:** Como *Administrador del Refugio*, quiero cambiar el estado operativo de una mascota para controlar en tiempo real su disponibilidad para adopción. **(Asociado a RF06)**
- **HU03:** Como *Veterinario*, quiero registrar las vacunas aplicadas para mantener el historial clínico actualizado y auditable. **(Asociado a RF15)**
- **HU04:** Como *Veterinario*, quiero consultar el cronograma de vacunas pendientes para programar oportunamente las próximas dosis preventivas. **(Asociado a RF16)**
- **HU05:** Como *Veterinario*, quiero registrar el inicio de un tratamiento médico para realizar un seguimiento riguroso a la recuperación clínica del animal. **(Asociado a RF18)**
- **HU06:** Como *Encargado de Adopciones*, quiero registrar el perfil de un nuevo adoptante para iniciar su proceso de evaluación socioeconómica. **(Asociado a RF24)**
- **HU07:** Como *Encargado de Adopciones*, quiero actualizar el estado de validación de un adoptante para habilitarlo o bloquearlo en futuros procesos de adopción. **(Asociado a RF30)**
- **HU08:** Como *Encargado de Adopciones*, quiero vincular formalmente una mascota disponible con un adoptante aprobado para concretar y registrar el contrato de adopción. **(Asociado a RF35)**
- **HU09:** Como *Administrador del Refugio*, quiero registrar las observaciones del seguimiento post-adopción para garantizar el bienestar continuo del animal en su nuevo entorno. **(Asociado a RF37)**
- **HU10:** Como *Usuario del Sistema*, quiero autenticarme mediante credenciales para acceder de forma segura a los módulos según mis permisos. **(Asociado a RF39)**

8. Diagrama de Clases

El diseño arquitectónico del sistema se fundamenta en el modelado de dominio, abstrayendo los procesos reales del refugio en entidades de software mediante la Programación Orientada a Objetos. Se ha priorizado el principio de Responsabilidad Única (SRP), garantizando que cada clase gestione de forma independiente su estado y comportamiento.

A continuación, se detalla la descripción estructural de las clases principales del sistema:

- **Mascota:** Entidad central. Encapsula la información intrínseca del animal rescatado (especie, raza, edad) y su estado operativo (disponible, en tratamiento, adoptado).
- **Adoptante:** Representa el perfil del candidato. Almacena de forma segura los datos personales, de contacto y el estado de validación socioeconómica.
- **CitaMedica:** Entidad clínica transaccional. Aísla la lógica de las revisiones veterinarias, registrando fecha, hora, diagnóstico y control de peso/talla.
- **Tratamiento:** Modela las prescripciones prolongadas. Permite trazabilidad desde su inicio hasta su finalización, vinculándose al historial de una mascota.
- **Vacuna:** Encapsula el control del esquema de inmunización (dosis aplicadas y pendientes), permitiendo al sistema generar alertas preventivas.
- **Adopcion:** Clase transaccional principal. Materializa la vinculación entre un objeto Mascota y un objeto Adoptante, encapsulando el contrato y seguimiento.
- **Usuario:** Entidad de seguridad. Define credenciales (usuario, contraseña y rol) del personal encargado, asegurando la integridad del sistema.



9. Criterios de Aceptación

Para garantizar que el software cumpla con los estándares de calidad esperados, se han definido los siguientes criterios de aceptación estructurados bajo el formato **BDD (Behavior-Driven Development)** mediante la sintaxis *Dado/Cuando/Entonces*. Estos criterios determinan las condiciones exactas que debe cumplir cada Historia de Usuario para considerarse terminada.

Criterios para HU01: Registro de nueva mascota (Asociado a RF01)

- **Criterio 1 (Ruta Feliz):**
 - **Dado que** el personal está en el formulario de "Registro de Mascota".
 - **Cuando** ingresa todos los datos obligatorios correctamente (Especie, Raza, Nombre) y presiona "Guardar".
 - **Entonces** el sistema persiste la información en el archivo local, muestra el mensaje "Mascota registrada con éxito" y limpia el formulario.
- **Criterio 2 (Error de validación):**
 - **Dado que** el personal está en el formulario de "Registro de Mascota".
 - **Cuando** deja el campo "Especie" vacío y presiona "Guardar".
 - **Entonces** el sistema bloquea el guardado y muestra una alerta indicando "El campo Especie es obligatorio".

Criterios para HU02: Cambio de estado de la mascota (Asociado a RF06)

- **Criterio 1:**
 - **Dado que** una mascota se encuentra con el estado "Disponible".
 - **Cuando** el administrador cambia su estado a "En Tratamiento" desde la vista de gestión.
 - **Entonces** el sistema actualiza el registro y la mascota deja de aparecer automáticamente en los listados de adopción.

Criterios para HU06 y HU07: Gestión y validación de Adoptantes (Asociado a RF24 y RF30)

- **Criterio 1 (Validación de duplicados):**
 - **Dado que** el encargado intenta registrar a un nuevo adoptante.
 - **Cuando** ingresa un DNI que ya existe en los registros de archivos del sistema.
 - **Entonces** el sistema rechaza la operación y muestra el mensaje "El DNI ingresado ya se encuentra registrado".
- **Criterio 2 (Rechazo justificado):**
 - **Dado que** un adoptante está en estado "En evaluación".
 - **Cuando** el administrador cambia el estado a "Rechazado" sin ingresar un motivo en el campo de texto.
 - **Entonces** el sistema impide el cambio de estado exigiendo el ingreso obligatorio del motivo.

Criterios para HU08: Proceso de vinculación y adopción (Asociado a RF35)

- **Criterio 1 (Vinculación exitosa):**
 - **Dado que** el administrador está en el módulo de Adopciones.
 - **Cuando** selecciona a un adoptante con estado "Aprobado" y lo vincula a una mascota con estado "Disponible", confirmando la acción.
 - **Entonces** el sistema registra el contrato, cambia el estado de la mascota a "Adoptada" de forma automática y bloquea la mascota para futuras solicitudes.

Criterios para HU10: Autenticación de Usuario (Asociado a RF39)

<<Colocar los apellidos e inicial del primer nombre de los autores en orden alfabético, separados por punto y coma>>

- **Criterio 1 (Acceso denegado):**

- **Dado que** un usuario intenta acceder al sistema.
- **Cuando** ingresa un nombre de usuario o contraseña incorrectos y hace clic en "Ingresar".
- **Entonces** el sistema deniega el acceso, no carga ningún módulo y muestra una alerta de "Credenciales inválidas".

10. Implementación del Proyecto

La fase de construcción del software se ha ejecutado respetando los lineamientos de la Programación Orientada a Objetos y las restricciones técnicas establecidas, garantizando un código limpio y modular.

10.1 Tecnologías utilizadas

Para garantizar un entorno de desarrollo estandarizado y alineado a los objetivos académicos, el ecosistema tecnológico se compone de:

- **Java SE (Standard Edition):** Lenguaje base del proyecto. Se ha utilizado su núcleo puro para aplicar encapsulamiento, polimorfismo y herencia sin depender de *frameworks* de terceros.
- **Eclipse IDE:** Entorno de Desarrollo Integrado seleccionado por su robustez en la compilación y depuración de proyectos nativos en Java.
- **Git y GitHub:** Herramientas núcleo para el control de versiones colaborativo. Durante esta fase no solo se utilizaron comandos básicos (add, commit, push), sino que se integraron flujos avanzados requeridos para la gestión de ramas y resolución de errores, tales como git stash (para guardar cambios temporales sin hacer commit), git revert (para deshacer cambios de forma segura en el historial) y git cherry-pick (para trasladar *commits* específicos entre ramas).

10.2 Manejo de archivos en Java

Dada la restricción operativa de no integrar motores de bases de datos relacionales, la persistencia de la información del refugio se diseñó utilizando el paquete nativo java.io. La implementación se basa en la **Serialización de Objetos** (mediante ObjectOutputStream y ObjectInputStream). Esto permite que el estado completo de las colecciones (listas de Mascotas, Adoptantes, Citas, etc.) se convierta en una secuencia de bytes y se guarde localmente en archivos con extensión .dat o .txt. Al reiniciar el sistema, estos archivos se leen (deserializan), reconstruyendo los objetos en memoria exactamente como quedaron en la sesión anterior, garantizando así la integridad y trazabilidad de los registros.

10.3 Funcionalidades implementadas

(Nota para el avance de Semanas 3 y 4): En la iteración actual del ciclo de desarrollo, la implementación se ha centrado en la base arquitectónica y de repositorios:

- Creación del repositorio remoto y estructuración de ramas (GitFlow).
- Implementación en código de las clases maestras (Mascota, Adoptante, CitaMedica, Tratamiento, Vacuna, Adopcion, Usuario) con sus respectivos atributos encapsulados (métodos *getters* y *setters*).
- Desarrollo de los constructores base de cada clase, respetando el diagrama de dominio propuesto.
- (Para las entregas finales, aquí se añadirán las pantallas y botones específicos que vayas programando).

11. Resultados

12. Conclusiones

<<Colocar los apellidos e inicial del primer nombre de los autores en orden alfabético, separados por punto y coma>>

13. Bibliografía

- Animals First. (s. f.). *Animal shelter software for rescues & shelters*. <https://animalsfirst.com/>
- Association of Shelter Veterinarians. (2022). *Guidelines for standards of care in animal shelters*. <https://www.sheltervet.org/guidelines-for-standards-of-care-in-animal-shelters>
- Capterra. (2026). *Best animal shelter software 2026*. <https://www.capterra.com/animal-shelter-software/>
- HumanePro. (s. f.). *Rescue group best practices: Recordkeeping*. <https://humanepro.org/page/rescue-group-best-practices-recordkeeping>

14. Anexos